

Exercise 7 - Summary Comparison

1. Comparison Table

Program	Collectives Used	Array Allocation	Manual Sum on Root?	Result Available On
sum_bcast	Bcast, Send, Recv	Full array everywhere	Yes	Root only
sum_scatter	Scatter, Send, Recv	Full on root only, chunk elsewhere	Yes	Root only
sum_gather	Scatter, Gather	Full on root only, chunk elsewhere	Yes	Root only
sum_reduce	Scatter, Reduce	Full on root only, chunk elsewhere	No	Root only
sum_allreduce	Scatter, Allreduce	Full on root only, chunk elsewhere	No	All processes
sum_scan	Scatter, Scan	Full on root only, chunk elsewhere	No	Different per rank

2. Timing Results (seconds)

Program	np = 2	np = 4	np = 8
sum_bcast	0.0061	0.0128	0.0162
sum_scatter	0.0061	0.0048	0.0046
sum_gather	0.0059	0.0045	0.0064
sum_reduce	0.0043	0.0049	0.0051
sum_allreduce	0.0062	0.0048	0.0048
sum_scan	0.0132	0.0114	0.0119

Bcast gets slower as process count increases, since the full 1,000,000 element array is duplicated to every process more processes means more total data copied. Scatter, Gather, Reduce and Allreduce stay roughly flat or even improve slightly, since each process only handles its own small chunk regardless of process count. Scan is consistently the slowest of the non-Bcast approaches, since it maintains a running cumulative total across ranks rather than a single combined result.

Overall, Reduce and Allreduce were the fastest and most consistent across process counts, since they combine collection and computation into one efficient call without duplicating the full array.

3. Thinking Question: When to Choose Scan over Allreduce

Allreduce gives every process the same final total, useful when all processes just need the overall result. Scan is useful instead when each process needs to know its own position or offset within a combined result, not just the final total.

Example: splitting a large file for parallel writing, where each process needs to know where in the file its own chunk starts. Using Scan, each process can compute `sum_before_me` (the total size of all chunks before it) and use that directly as its starting write offset, without needing any extra communication.